Project 1

CIFAR-10 图像分类：卷积网络的实现与实验

github.com/yuxuanli1103

2026 年 10 月

摘要

本项目在 CIFAR-10 上从零实现了一套可复现的图像分类流程：自建数据管线（固定 9:1 划分、干净与增强双视图）、一个三层卷积网络及其全连接基线、统一的训练/评估/汇总脚本。在此之上设计并完成了五组对照实验，分别考察优化器与学习率、模型容量与归纳偏置、归一化与 Dropout 与初始化、训练长度带来的过拟合、以及学习率衰减策略，共 26 次训练。

主要结果：基线（Adam 10^{-3}，20 轮）测试准确率 0.8085；最好模型（SGD 0.05 + 余弦衰减，30 轮）测试准确率 **0.8385**，宏平均 F1 0.8381。等参数量的 MLP 在训练集上就停在 0.45 附近，比 CNN 低约 34 个百分点，且加宽参数没有任何改善——这是全部实验中最强的一条证据，说明卷积结构携带的局部性与平移不变先验是不可由样本量简单替代的。控制训练轮数后可以看到，12 轮时全部配置处于欠拟合区，正则化项无从体现；把预算拉长到 40 轮才出现可测的过拟合（gap 达 0.080），此时 Dropout 0.5 把 gap 压到 0.037，而 BN 的作用是“同样的训练准确率、更小的 gap”。

关键词：CIFAR-10；卷积神经网络；归纳偏置；批归一化；Dropout；学习率衰减

目录

1 实现 **3**

1.1 代码结构 3
1.2 数据管线 3
1.3 模型 4
1.4 训练循环 4
1.5 评价指标 5
1.6 实验编排与汇总 5

2 实验设计 **5**

2.1 固定设置 5
2.2 五组实验 6
2.3 指标口径与显著性 6

3 结果 **6**

3.1 基线与最终模型 6
3.2 超参数调优 7
3.3 容量与归纳偏置 8
3.4 归一化、Dropout 与初始化 9
3.5 过拟合与欠拟合：训练长度的影响 10
3.6 学习率衰减 11
3.7 结果汇总 12

A 完整运行清单 **13**

B 复现命令 **14**

1 实现

1.1 代码结构

全部代码共 7 个 Python 脚本、861 行。模块之间是单向依赖：`train.py` 依赖 `dataset.py`、`model.py`、`metrics.py`；实验编排 `experiments.py` 只通过子进程调用 `train.py`；出图脚本 `analyze.py` 只读取磁盘上的结果文件。这种“结果落盘、下游只读”的组织方式使得任一组实验可以单独重跑而不影响其他组。

表 1: 模块清单与职责

文件	行数	职责
`dataset.py`	111	读取 pickle、9:1 划分、增强与归一化、返回 4 个 DataLoader
`model.py`	116	`CNN` 与 `MLP`、归一化层开关、四种初始化、参数量统计
`metrics.py`	43	准确率、宏平均精确率/召回率/F1、混淆矩阵
`train.py`	208	训练与评估循环、优化器/调度器工厂、checkpoint 与历史记录
`evaluate.py`	89	用指定 checkpoint 在 val 或 test 上评估，写 JSON
`experiments.py`	109	五组实验的配置表与批量子进程编排
`analyze.py`	185	读 `experiments.json`，出表与出图

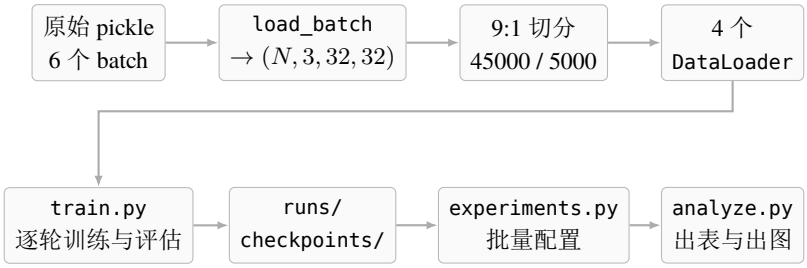


图 1: 数据与产物在模块之间的流向。上排是数据准备，下排是训练与汇总；所有跨模块的传递都通过磁盘文件完成。

1.2 数据管线

CIFAR-10 的原始文件是 Python 2 时代序列化的 pickle。每个 batch 里 `data` 是 $N \times 3072$ 的 `uint8` 矩阵（行优先展开的 $3 \times 32 \times 32$），`labels` 是 N 个整数。读取时需要显式指定 `encoding="latin1"`，否则 Python 3 会在非 ASCII 字节上失败；随后按 PyTorch 的 NCHW 约定重排：

```python
def load_batch(name):
    with open(DATA_ROOT / name, "rb") as f:
        d = pickle.load(f, encoding="latin1")
    images = torch.from_numpy(d["data"]).reshape(-1, 3, 32, 32)
    return images, torch.tensor(d["labels"])
```

划分方式对结论的可比性至关重要。基线、调参、容量等所有实验必须面对**同一批**训练样本与验证样本，否则组间差异会混入划分噪声。因此划分不用 `random.shuffle`，而用带固定种子的置换：

```python
perm = torch.randperm(len(labels), generator=torch.Generator().manual_seed(seed))
n_val = int(len(labels) * val_ratio)
val_idx, train_idx = perm[:n_val], perm[n_val:]
```

`seed` 默认为 0，因此无论何时运行，前 5000 个置换位置永远是验证集。最终得到 train 45000 / val 5000 / test 10000。

数据增强只出现在训练视图上：先随机裁剪（32 像素图像四周各补 4 像素后随机取 32×32），再以 0.5 概率水平翻转，最后转成 `float32` 并按通道归一化。归一化常数取训练集统计量 $\mu = (0.4914, 0.4822, 0.4465)$、$\sigma = (0.2470, 0.2435, 0.2616)$。评估视图只做后两步。

值得注意的是 `get_loaders` 返回**四个**加载器，第四个 `train_eval_loader` 与训练集是同一批索引，但使用不带增强的评估变换。它的用途在第 2.3 节说明；这里只强调一点：训练循环每一轮都会在它上面再跑一遍，换来一个与验证集可直接相减的训练集准确率。

1.3 模型

卷积部分由三个相同的块堆叠而成，每块是 3×3 卷积（padding 1，保持空间尺寸）$\to$ 归一化 $\to$ ReLU $\to$ 2×2 最大池化。三个块把 $32 \to 16 \to 8 \to 4$，通道数按 width 倍数取 $32, 64, 128$。

```
def conv_block(c_in, c_out, norm):
    return nn.Sequential(
        nn.Conv2d(c_in, c_out, 3, padding=1),
        make_norm(norm, c_out),
        nn.ReLU(),
        nn.MaxPool2d(2),
    )
```

分类头把 $128 \times 4 \times 4 = 2048$ 维展平，接一个 256 维隐藏层与 10 维输出，中间插一层 Dropout。归一化层通过 `make_norm` 工厂返回 `BatchNorm2d` 或 `Identity`，这样“有 BN / 无 BN”只是一个命令行参数，而不是两份代码。

作为对照的 MLP 把 $3 \times 32 \times 32$ 直接展平成 3072 维，接 $512w \to 256w \to 10$ 的两层隐藏层，第一层后接 Dropout。两条分支参数量相差不大，甚至 MLP 更大：

表 2: 四种配置的可训练参数量（$\times$ 为 width 倍数）

模型	参数量	相对 CNN width 1	主要构成
CNN width 1	620,810	1.00×	卷积 93k + 分类头 527k
CNN width 2	1,423,114	2.29×	卷积 370k + 分类头 1,051k
MLP width 1	1,707,274	2.75×	$3072 \to 512$：1,573k
MLP width 2	3,676,682	5.92×	$3072 \to 1024$：3,147k

初始化有四种：PyTorch 默认（Kaiming uniform）、`kaiming_normal_`（fan_out，ReLU 增益）、`xavier_uniform_`、以及全零。偏置一律置零。

1.4 训练循环

一个 epoch 的函数把“训练”与“评估”统一到同一份代码里，两者只由一个布尔量区分。这个开关同时控制两件事：`model.train(train)` 切换 BN 统计量的更新与 Dropout 的开合，`torch.set_grad_enabled(train)` 决定是否建立计算图。

```
with torch.set_grad_enabled(train):
    logits = model(images)
    loss = criterion(logits, labels)
    if train:
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

损失的累加方式需要留意：`loss.item() * len(labels)` 累加成总损失，最后除以样本总数，得到的是按样本加权的 epoch 平均损失，而不是各 batch 损失的算术平均（后者会让最后一个不满的 batch 获得过高权重）。

优化器与调度器同样用工厂函数实现，参数名直接对应命令行开关：SGD 固定 momentum 0.9，Adam 用默认 β，AdamW 用 weight decay 5×10^{-4}；调度器有恒定的 `none`、按 epoch 余弦退火的 `cosine`（T_{max} 取总轮数）、以及每 1/3 轮降 10 倍的 `step`。

每轮结束后依次保存：TensorBoard 标量（三组前缀 `train/`、`val/`、`train_clean/`）、一条完整的 JSON 历史记录（含当轮学习率）、`_last.pt`，以及当验证准确率刷新纪录时的 `_best.pt`。checkpoint 里存 `model.state_dict()`、轮次、最优准确率和完整超参数，因此 `evaluate.py` 可以只凭一个 `.pt` 文件重建出结构完全一致的模型。

1.5 评价指标

除准确率外还报告宏平均的精确率、召回率和 F1。CIFAR-10 每类恰好 1000 张测试样本，宏平均与加权平均数值差别很小，但宏平均对“某一类被整体漏判”更敏感——若某类召回率为 0，宏平均会直接掉 1/10，而加权平均只掉该类占比。

```
precision, recall, f1, _ = precision_recall_fscore_support(
    y_true, y_pred, labels=range(num_classes),
    average="macro", zero_division=0,
)
```

`labels=range(10)` 保证即使某一类一次都没被预测到，输出仍是 10 维而不会缩水。混淆矩阵由 `confusion_matrix` 给出，`evaluate.py` 额外报告每一类的对角占比，即该类的单类准确率。

1.6 实验编排与汇总

`experiments.py` 为每组实验维护一张配置表，逐条构造命令行并 `subprocess.run` 一个独立进程。用子进程而不是在同一进程内循环调用 `main()`，是为了让每次运行的全局随机状态、CUDA 显存和 `torch.manual_seed` 都从干净状态开始，避免前一次运行的残留影响后一次。每次运行结束后读回自己的历史 JSON，抽取最优验证准确率、末轮训练/验证准确率、以及 gap 与耗时，追加进 `runs/experiments.json` 的分组键下。

`analyze.py` 按组选择绘图函数：调参组画最优验证准确率的柱状图，容量组画“末轮 train/val 柱状图 + 训练准确率曲线”，正则化组画“末轮准确率 + gap”，过拟合组画“训练/验证双曲线 + gap 随轮次变化”，衰减组画“验证准确率 + 学习率曲线”。所有组统一再出一张验证集 accuracy/loss 的曲线图。

2 实验设计

2.1 固定设置

除被考察的变量外，所有运行固定为：CIFAR-10，train 45000 / val 5000 / test 10000，`seed=0`，batch 128，`num_workers=0`，随机裁剪 + 水平翻转的增强，上述均值方差归一化。除特别说明，模型为 CNN width 1、BN、Dropout 0.3、默认初始化。

2.2 五组实验

每个实验组只改变一个维度。轮数是由预算决定的：调参、容量、正则化三组用 12 轮；过拟合组必须足够长才能让 gap 显现，用 40 轮；衰减组要让衰减走完一个完整周期，用 30 轮。衰减组还特意配了同轮数、同超参、只关衰减的对照组，否则无法区分“衰减有用”与“单纯练得久”。

表 3: 五组实验的变量与预算

组	考察变量	取值	运行数	轮数
调参	优化器 × 学习率	Adam $\{10^{-4}, 10^{-3}, 10^{-2}\}$、SGD $\{10^{-2}, 5\times10^{-2}\}$、AdamW 3×10^{-4}	6	12
容量	结构 × width	{CNN, MLP} × {1, 2}	4	12
正则化	BN / Dropout / 初始化	7 种组合，含 zero init	7	12
过拟合	同上，但训练拉长	{无正则, BN, BN+Dropout 0.5}	3	40
衰减	调度器	SGD 与 Adam 各自的固定/余弦，另加 SGD+step	5	30
基线	——	Adam 10^{-3}，默认结构	1	20

2.3 指标口径与显著性

报告中的 `train` 一律指**干净数据（无增强）**上的训练集准确率，即第四份加载器给出的 `train_clean`。原因很直接：随机裁剪与水平翻转本身会制造“错误”的训练样本（一张被裁掉一半的船可能看起来像飞机），用增强视图上的准确率与验证准确率相减，gap 甚至可能为负，读不出任何信息。这一点在实现上通过“同一批索引、两套变换”保证了两者严格可比。

gap 定义为 `train_clean` 减验证准确率。

验证集只有 5000 张，准确率的标准误约为 $\sqrt{0.8\times0.2/5000}\approx0.0057$，95% 置信区间半宽约 ±0.010。因此组间差距小于 0.02 的结果在本报告中一律不作为真实差异来讨论；超过 0.02 的差异才进入结论。

3 结果

3.1 基线与最终模型

基线配置（CNN width 1、BN、Dropout 0.3、Adam 10^{-3}、20 轮、无衰减）在验证集上的最优准确率为 0.8110，出现在最后一轮。用该 checkpoint 在测试集上评估，准确率 0.8085。把预算拉到 30 轮、换成 SGD 0.05 加余弦衰减，测试准确率提高到 0.8385，宏平均 F1 从 0.8092 升到 0.8381。

表 4: 测试集结果对比（各取最优验证轮次的 checkpoint）

指标	基线（Adam，20 轮）	SGD + 余弦（30 轮）	变化
accuracy	0.8085	**0.8385**	+3.00 pp
precision (macro)	0.8118	0.8382	+2.64 pp
recall (macro)	0.8085	0.8385	+3.00 pp
F1 (macro)	0.8092	0.8381	+2.89 pp
loss	0.5520	0.4719	−0.0801

分类型看，提升相当均匀：十个类别中有九类上涨，幅度从狗、卡车的 +1.4 个百分点到马

的 +6.3 个百分点。只有鸟下降 0.8 个百分点，在 1000 张测试样本的量级上约等于 8 张图，属于噪声范围。

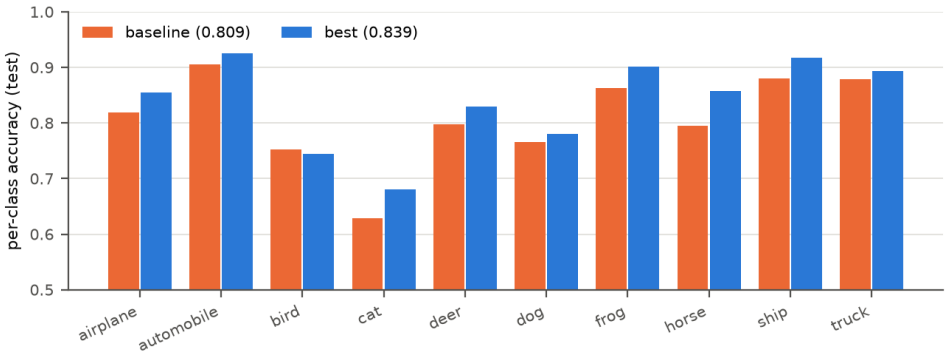


图 2: 测试集单类准确率。最好的两个类别是汽车（0.925）与船（0.918），最差的是猫（0.680），其次是鸟（0.745）。这一排序在基线中就已经存在，30 轮余弦衰减没有改变它。

混淆结构同样稳定。两个模型最大的两对混淆都是“猫 → 狗”和“狗 → 猫”，合计约占全部错误的 15%（基线 280/1915 张，最终模型 245/1615 张）。后续几对在两者之间有些换位：基线里是“马 → 狗”“飞机 → 鸟”，最终模型里是“鸟 → 鹿”“鸟 → 蛙”。这些类别在 32×32 分辨率下共享大量纹理与轮廓，是 CIFAR-10 的典型失败模式。

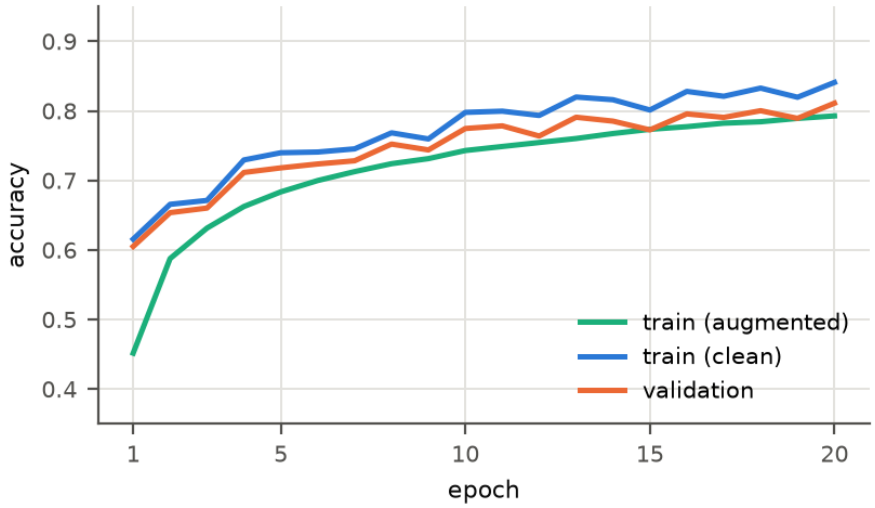


图 3: 基线训练过程。增强视图上的训练准确率（绿）始终低于干净训练集（蓝），二者在末轮相差 4.8 个百分点——这正是增强带来的“假性错误”。验证准确率（橙）末轮为 0.811，与干净训练准确率相差 3.0 个百分点，20 轮时尚无明显的验证损失回升。

3.2 超参数调优

六组配置各跑 12 轮。最优配置是 SGD 10^{-2}（0.7930），最差是 Adam 10^{-2}（0.6910），跨度超过 10 个百分点，而学习率的作用远大于优化器种类本身。

学习率呈现清晰的倒 U 型：10^{-4} 时 12 轮走不完，10^{-2} 时第 1 轮只有 0.27、其后 11 轮挣扎到 0.69，是步长过大的典型形态。AdamW 的 weight decay 5×10^{-4} 在这组里没有优势，因为模型远未过拟合，权重衰减没有可压制的东西。

12 轮预算下 SGD 略胜 Adam，但这并不是“SGD 更好”的证据——第 3.6 节把预算拉到 30 轮、两者都用固定学习率，Adam 反而领先 1.1 个百分点。优化器的排名不能脱离训练预算来谈。

表 5: 调参组（12 轮）。`train` 为末轮干净训练集准确率，`val` 为末轮验证准确率

配置	best val	train	val	gap
SGD 10^{-2}	**0.7930**	0.8064	0.7820	+0.0244
Adam 10^{-3}	0.7854	0.7931	0.7642	+0.0289
AdamW 3×10^{-4}	0.7766	0.7823	0.7648	+0.0175
SGD 5×10^{-2}	0.7710	0.7816	0.7562	+0.0254
Adam 10^{-4}	0.7294	0.7487	0.7294	+0.0193
Adam 10^{-2}	0.6910	0.7001	0.6910	+0.0091

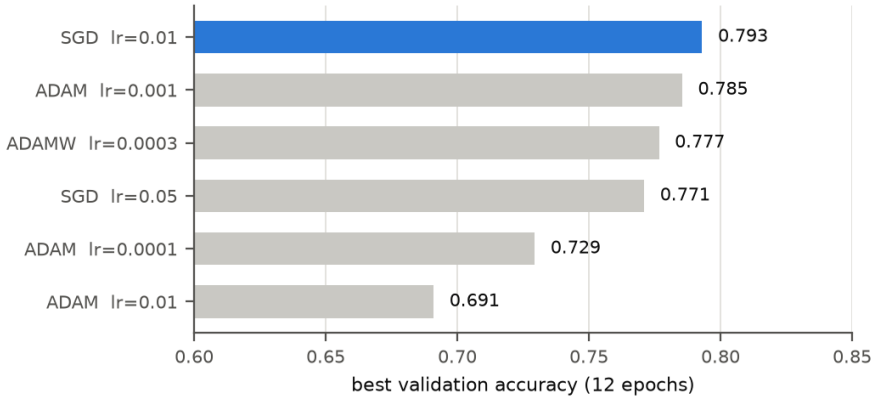


图 4: 调参组的最优验证准确率。同为 Adam，学习率从 10^{-4} 到 10^{-3} 再到 10^{-2}，准确率先升后降（倒 U 型）。

3.3 容量与归纳偏置

四种配置各跑 12 轮，参数量从 62 万到 368 万。

表 6: 容量组（12 轮）

配置	参数量	best val	train	val
CNN width 2	1,423,114	**0.7968**	0.7989	0.7746
CNN width 1	620,810	0.7798	0.8019	0.7766
MLP width 1	1,707,274	0.4478	0.4585	0.4456
MLP width 2	3,676,682	0.4412	0.4460	0.4412

这是全部实验里最干净的一组结论。MLP 的参数量是 CNN 的 2.7 至 6 倍，准确率却只有 0.44 对 0.78，差 34 个百分点。更关键的是 MLP 的**训练集**准确率也只有 0.45–0.46：它在训练数据上就学不动，是欠拟合，而非过拟合。把 MLP 从 width 1 加宽到 width 2，参数翻倍到 368 万，末轮验证准确率从 0.4456 变成 0.4412，完全在噪声之内。

两条曲线的形状比终值更能说明问题。CNN 第 1 轮的干净训练准确率是 0.62，MLP 只有 0.39；CNN 在 12 轮内爬到 0.80，MLP 则卡在 0.46 附近近乎平坦。增加宽度对 MLP 没有任何帮助——瓶颈不在参数数量，而在结构本身：全连接层把 $3 \times 32 \times 32$ 展平成一维，第 0 个特征与相邻像素、与相隔 32 个位置的像素被同等对待，空间结构必须完全由数据学回来，而 45000 张样本远远不够。卷积层用一个小窗口滑过全图，等于把局部性与平移不变两个先验直接写进了结构。

CNN 内部的加宽同样收益有限：620K 到 1.42M，末轮验证准确率 0.7766 对 0.7746，反而略低；最优验证准确率 0.7798 对 0.7968 则相反。两个方向的差异都在 ± 0.02 以内，说明 width 1 的容量对 12 轮预算下的 CIFAR-10 已经够用，此时的瓶颈是训练长度，不是容量。

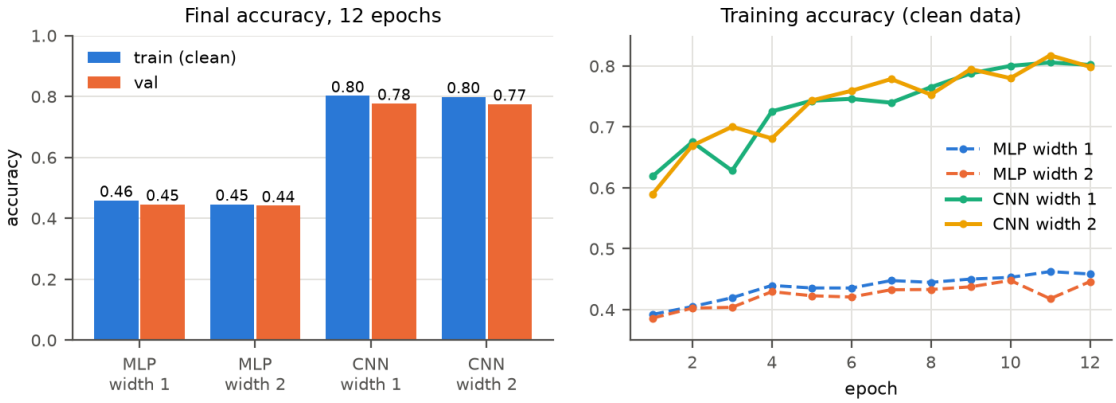


图 5: 左：末轮训练集（干净）与验证集准确率；右：干净训练准确率随轮次的变化。MLP 的两条曲线全程贴着 0.40–0.47 走，12 轮结束仍在缓慢上升但斜率已接近水平；CNN 第 1 轮就到 0.62，随后持续爬升。

3.4 归一化、Dropout 与初始化

七种配置各跑 12 轮，全部使用 CNN width 1。

表 7: 正则化组（12 轮）

配置	best val	train	val	gap
BN only	**0.7910**	0.8050	0.7774	+0.0276
BN + Dropout 0.3	0.7846	0.8010	0.7714	+0.0296
无 BN 无 Dropout	0.7838	**0.8244**	0.7838	+0.0406
Dropout 0.5 only	0.7606	0.7921	0.7606	+0.0315
BN + Dropout + Kaiming	0.7604	0.7789	0.7604	+0.0185
BN + Dropout + Xavier	0.7578	0.7783	0.7578	+0.0205
BN + Dropout + 全零初始化	0.1022	0.1006	0.0944	+0.0062

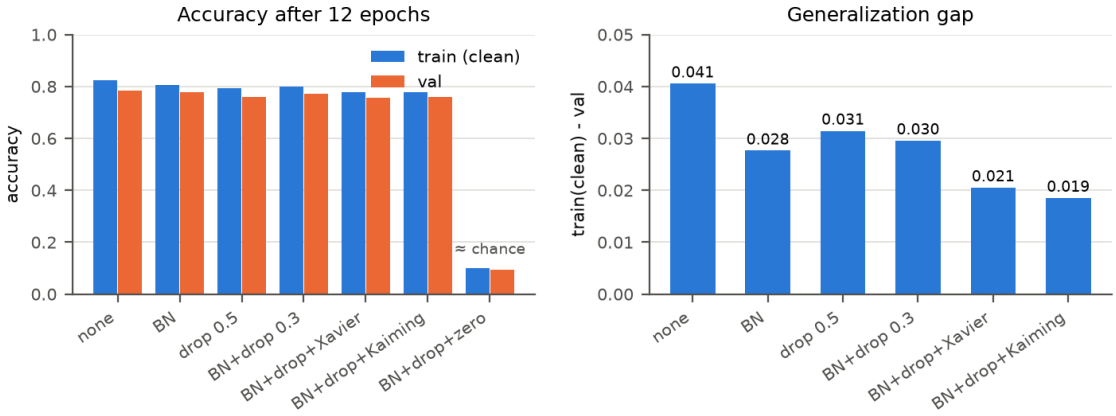


图 6: 左：12 轮后的训练（干净）与验证准确率；右：gap（零初始化因整体失效而略去）。除零初始化外，所有配置的验证准确率都挤在 0.76–0.79 之间。

除零初始化外，所有配置的验证准确率都在 0.76–0.79 之间，彼此差异不超过 0.03，基本落在 ±0.01 置信区间的边缘。这一组在 12 轮下真正有信号的是 gap 列：无正则配置的 gap 最大（0.0406），它的训练准确率也最高（0.8244）。换句话说，正则化确实压住了 gap，但那是因为它先把训练准确率压低了，验证准确率并没有因此变好。这正是欠拟合区的特征——还不需要防过拟合。要看这组变量的真实作用，必须把训练拉长。

零初始化是唯一给出绝对结论的配置：准确率 0.10，等于随机猜。所有权重为零时，同一层

所有神经元的梯度完全相同、永远同步更新，网络退化成单个神经元，“对称性无法被打破”。这解释了为什么框架的默认初始化不是零。

Kaiming 与 Xavier 反而低于默认初始化（0.760 和 0.758 对 0.785），原因是在 12 轮预算下这两个分布把初期收敛拖慢了；配合 BN 时默认初始化已经足够合理。初始化方案的好坏没有脱离网络深度与是否使用 BN 的绝对答案。

3.5 过拟合与欠拟合：训练长度的影响

把训练拉长到 40 轮，只改正则化配置。

表 8: 过拟合组（40 轮）

配置	best val（轮次）	末轮 train	末轮 val	末轮 gap
BN only	**0.8440**（38）	0.8963	0.8334	+0.0629
BN + Dropout 0.5	0.8382（39）	0.8669	0.8304	+0.0365
无 BN 无 Dropout	0.8300（39）	0.8970	0.8174	+0.0796

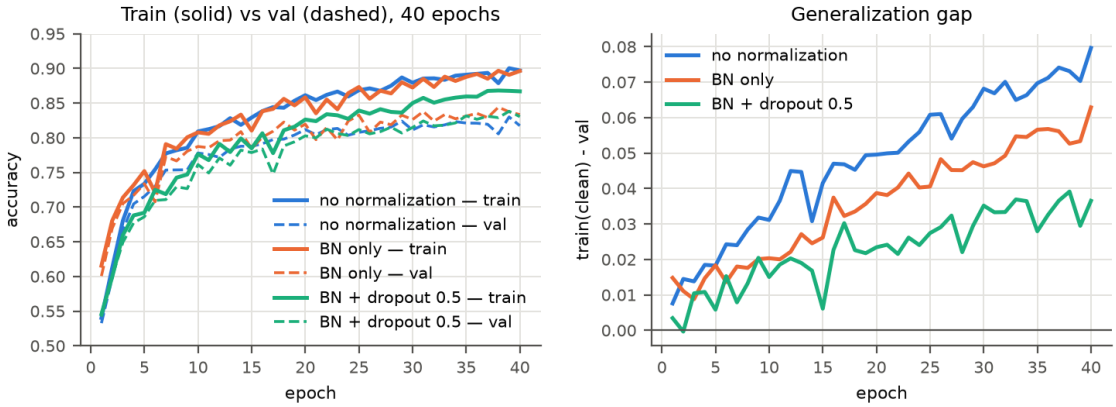


图 7: 左：训练（实线）与验证（虚线）准确率；右：gap 随轮次的变化。gap 在三种配置下都持续扩大，只是斜率不同；无正则那条在中段有几次回摆。

与 12 轮时相比，结论发生了质变。40 轮结束时训练准确率已达到 0.87–0.90，验证准确率停在 0.82–0.83，gap 从 10 轮时的 0.015–0.031 扩大到 0.037–0.080。第 10 轮时三种配置的 gap 都小于 0.032，验证准确率还在涨；之后训练准确率继续上升而验证准确率趋于平台，gap 单向扩大。同时验证损失开始触底回升：无正则配置在第 39 轮达到最低 0.5148，之后回到 0.548；BN 配置在第 38 轮最低 0.4637，之后回到 0.496。两条判据——训练准确率是否还在涨、验证损失是否触底回升——同时成立，可以判定进入了轻度过拟合。

Dropout 是这里唯一有效的正则化手段。0.5 的 Dropout 把末轮 gap 从 0.080 压到 0.037，压掉一半以上；代价是训练准确率从 0.897 降到 0.867（−3.0 个百分点），换回验证准确率 +1.3 个百分点（0.8174 → 0.8304）。这是教科书式的正则化权衡。

BN 的贡献形式不同。它与无正则配置的末轮训练准确率几乎相同（0.8963 对 0.8970），但验证准确率高出 1.6 个百分点、gap 小 1.7 个百分点。这个优势并非全程存在：第 1 轮 BN 的验证准确率领先 6.8 个百分点（0.601 对 0.533），第 15 轮反而落后 0.3 个百分点，到第 25 轮才重新拉开 2.5 个百分点并保持到结束。用 BN 把 gap 从 0.080 压到 0.063，仍不足以替代 Dropout。

还有一个直接的工程结论：三种配置的最优验证准确率都出现在第 38–39 轮，而不是最后一轮。这正是训练循环要保存 `_best.pt` 而不是只存末轮 checkpoint 的实际理由。

3.6 学习率衰减

五种配置各跑 30 轮，其中两种是“同预算、同超参、只关衰减”的对照组。

表 9: 衰减组（30 轮）

配置	best val（轮次）	末轮 train	末轮 val	gap	末轮学习率
SGD 0.05 + cosine	**0.8474**（30）	0.8880	0.8474	+0.0406	1.4×10^{-4}
Adam 10^{-3} 固定	0.8340（28）	0.8635	0.8252	+0.0383	1.0×10^{-3}
Adam 10^{-3} + cosine	0.8288（30）	0.8695	0.8288	+0.0407	2.7×10^{-6}
SGD 0.05 固定	0.8230（24）	0.8673	0.8204	+0.0469	5.0×10^{-2}
SGD 0.05 + step	0.8218（26）	0.8517	0.8204	+0.0313	5.0×10^{-4}

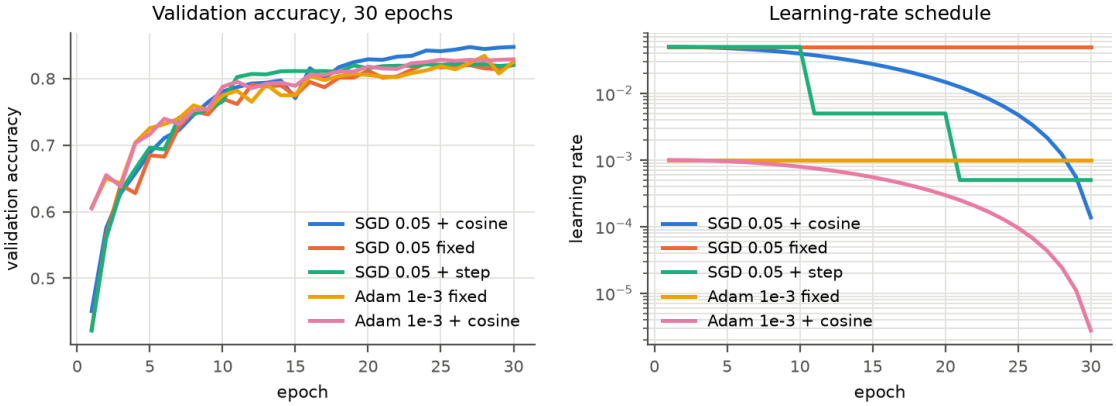


图 8: 左：30 轮的验证准确率曲线；右：实际学习率（对数坐标）。

余弦衰减对 SGD 有效：从固定学习率的 0.8230 升到 0.8474，涨 2.4 个百分点，而且曲线在第 20 轮之后仍在稳步上冲（末 10 轮再涨 1.9 个百分点）。对 Adam 则相反，从 0.8340 降到 0.8288。原因在右图可以直接读出来：Adam 的余弦衰减把末轮学习率压到 2.7×10^{-6}，比它的初始值小三个数量级，后期实际上等于停学；Adam 自身的逐参数自适应步长已经在做类似的“逐步收窄”，再叠加余弦衰减属于重复施力。

step 衰减不如余弦。SGD + step 的末轮学习率停在 5×10^{-4}，曲线在第 20 轮后基本走平；余弦在同一区间把学习率从 1.5×10^{-2} 连续降到 1.4×10^{-4}，验证准确率同时从 0.829 升到 0.847。衰减的**形状**比“有没有衰减”更重要。

这一组也纠正了调参组的误读：12 轮时 SGD 领先 Adam，30 轮固定学习率时 Adam 反超（0.8340 对 0.8230）。Adam 前期收敛快、后期需要更多轮次兑现，SGD 曲线更平直所以短预算下占便宜。

最后，30 轮余弦配置的 0.8474 已经超过 40 轮无正则配置的 0.8300，且只用了四分之三的轮数——先修优化比先堆轮数更划算。

3.7 结果汇总

- 最好单模型：SGD 0.05 + 余弦衰减、30 轮，测试准确率 0.8385、宏 F1 0.8381，比 20 轮 Adam 基线高 3.0 个百分点。
- 结构的作用远大于容量：等参数量的 MLP 比 CNN 低 34 个百分点，且表现为训练集上的欠拟合而非过拟合；加宽 MLP 无效。
- 12 轮的所有配置都处在欠拟合区，正则化项的正负效应均不可测；40 轮才出现 gap 持续扩大与验证损失回升。
- 40 轮下 Dropout 0.5 把 gap 从 0.080 压到 0.037，代价是训练准确率 -3.0 个百分点；BN 只把 gap 压到 0.063。
- 全零初始化使训练完全失效（0.10），验证了打破对称性的必要性；Kaiming 与 Xavier 在 12 轮预算下均不如默认初始化。
- 余弦衰减对 SGD 增益 $+2.4$ 个百分点，对 Adam 无益；衰减形状比有无衰减更重要。

A 完整运行清单

表 10: 全部 26 次运行。`train` 为末轮干净训练集准确率

运行名	轮数	best val	train	val	耗时 (s)
基线					
baseline_adam	20	0.8110	0.8409	0.8110	169
调参（12 轮）					
tuning_optimizersgd_lr0.01	12	0.7930	0.8064	0.7820	95
tuning_optimizeradam_lr0.001	12	0.7854	0.7931	0.7642	94
tuning_optimizeradamw_lr0.0003	12	0.7766	0.7823	0.7648	96
tuning_optimizersgd_lr0.05	12	0.7710	0.7816	0.7562	94
tuning_optimizeradam_lr0.0001	12	0.7294	0.7487	0.7294	98
tuning_optimizeradam_lr0.01	12	0.6910	0.7001	0.6910	96
容量（12 轮）					
capacity_modelcnn_width2	12	0.7968	0.7989	0.7746	120
capacity_modelcnn_width1	12	0.7798	0.8019	0.7766	94
capacity_modelmlp_width1	12	0.4478	0.4585	0.4456	91
capacity_modelmlp_width2	12	0.4412	0.4460	0.4412	80
正则化（12 轮）					
regularization_bn_only	12	0.7910	0.8050	0.7774	96
regularization_bn_drop	12	0.7846	0.8010	0.7714	107
regularization_no_norm_no_drop	12	0.7838	0.8244	0.7838	91
regularization_drop_only	12	0.7606	0.7921	0.7606	90
regularization_bn_drop_kaiming	12	0.7604	0.7789	0.7604	101
regularization_bn_drop_xavier	12	0.7578	0.7783	0.7578	94
regularization_bn_drop_zero_init	12	0.1022	0.1006	0.0944	97
过拟合（40 轮）					
overfit_long_bn	40	0.8440	0.8963	0.8334	320
overfit_long_bn_drop	40	0.8382	0.8669	0.8304	319
overfit_long_no_reg	40	0.8300	0.8970	0.8174	308
衰减（30 轮）					
schedule_sgd_cosine	30	0.8474	0.8880	0.8474	239
schedule_ctrl_adam_fixed	30	0.8340	0.8635	0.8252	241
schedule_adam_cosine	30	0.8288	0.8695	0.8288	266
schedule_ctrl_sgd_fixed	30	0.8230	0.8673	0.8204	237
schedule_sgd_step	30	0.8218	0.8517	0.8204	235

B 复现命令

```
# single run (default: CNN width 1, BN, dropout 0.3, Adam 1e-3, 20 epochs)
python train.py --epochs 20 --run-name baseline_adam

# the five experiment groups
python experiments.py --group tuning         --epochs 12
python experiments.py --group capacity       --epochs 12
python experiments.py --group regularization --epochs 12
python experiments.py --group overfit        --epochs 40
python experiments.py --group schedule       --epochs 30

# tables and figures
python analyze.py

# evaluate one checkpoint on the test split
python evaluate.py --ckpt checkpoints/schedule_sgd_cosine_best.pt --split test
```